KU LEUVEN

Natural Language Processing: Tokenization

H02B1A



Miryam de Lhoneux

Slides adapted from Jurafsky and Martin (2024)

Department of Computer Science
KU Leuven

Outline

Introduction

Regular expressions
- Basics
- Applications

Word tokenzation and normalization and sentence segmentation
- Word tokenization
- Word normalization
- Sentence segmentation

Subword tokenization
- BPE
- Subword tokenization: limitations

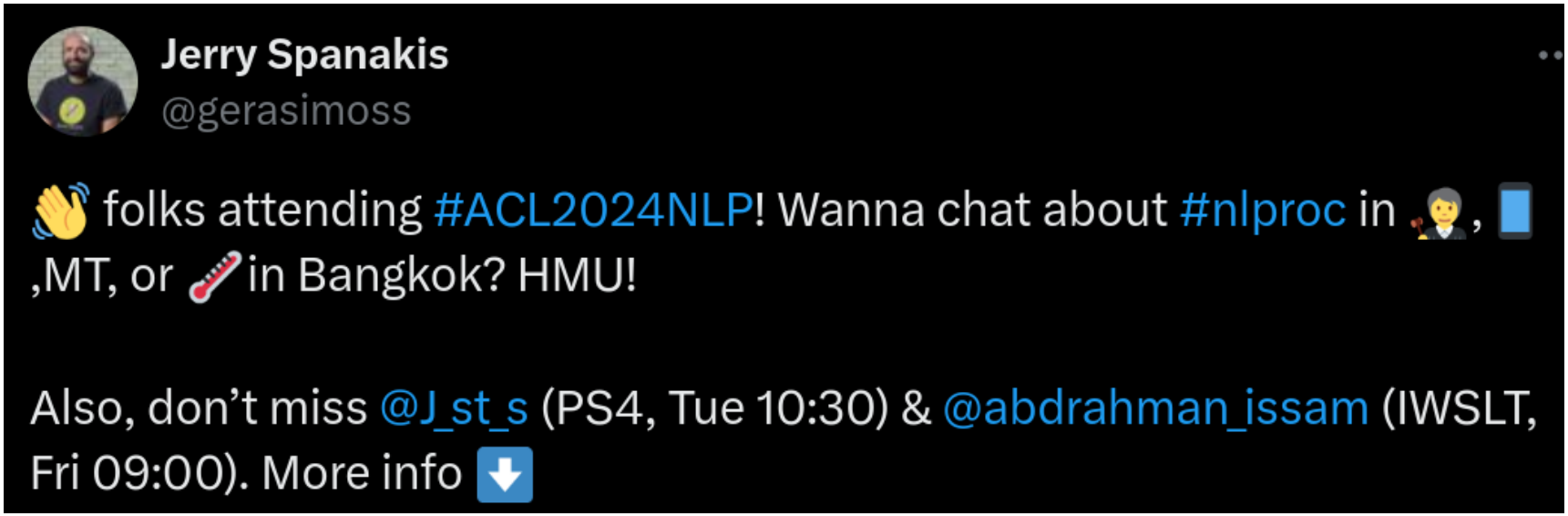
Summary

Vocabulary: {sky, wings, engine, helicopter, drone, rocket, goose, eagle, bee}

How do we determine what constitutes this vocabulary?

The goose is in the sky.
The eagle in the sky has wings.
The bee in the sky has wings.
The goose with wings in the sky.
The eagle with wings in the sky.
The bee with wings in the sky.
The goose and the eagle with wings are in the sky.
The goose flies in the sky.
The wings of the bee are good wings.
The helicopter, the drone and the rocket are in the sky.
The helicopter in the sky is a useful engine.
The drone in the sky is an engine that takes pictures.
A rocket is an engine.
The helicopter is an engine.
A drone is an engine that takes pictures.
A rocket is a useful engine among sky engines.
A helicopter is flying in the sky.
The sky is full of helicopters and drones.
The engine here is a drone, the one there is a rocket.

Toy corpus



Example tweet including emoji, hashtags, slang (*wanna*), abbreviations (*MT*, *HMU*)

Text is not always clean and may need a normalization step.


Jerry Spanakis
@gerasimoss
folks attending #ACL2024NLP! Wanna chat about #nlproc in , ,MT, or in Bangkok? HMU!
Also, don't miss @j_st_s (PS4, Tue 10:30) & @abdrahman_issam (IWSLT, Fri 09:00). More info
How can we normalize this text?
IT'S HOPELESS!
EVERYBODY STAND BACK.
I KNOW REGULAR EXPRESSIONS.
PERL!
TAP TAP

Outline

Introduction

Regular expressions
- Basics
- Applications

Word tokenzation and normalization and sentence segmentation
- Word tokenization
- Word normalization
- Sentence segmentation

Subword tokenization
- BPE
- Subword tokenization: limitations

Summary

Outline

Introduction

Regular expressions
- Basics
- Applications

Word tokenzation and normalization and sentence segmentation
- Word tokenization
- Word normalization
- Sentence segmentation

Subword tokenization
- BPE
- Subword tokenization: limitations

Summary

Regular expressions

Definition

A formal language for specifying text strings.

Regular expressions

How can we search for mentions of these cute animals in text?



- woodchuck
- woodchucks
- Woodchuck
- Woodchucks
- Groundhog
- groundhogs

Regular expressions: disjunctions

Pattern	Matches
`[wW]oodchuck`	Woodchuck, woodchuck
`[1234567890]`	Any one digit

Letters inside brackets

Pattern	Matches	Examples
`[0-9]`	A single digit	Chapter 1: Down the Hole
`[A-Z]`	An upper case letter	Drenched Blossoms
`[a-z]`	A lower case letter	my beans were impatient

Ranges using the dash

Regular expressions: Negation in disjunction

Pattern	Matches	Examples
[^A-Z]	Not an upper case letter	Drenched Blossoms
[^Ss]	Neither *S* nor *s*	my beans were impatient
[^.]	Not a period	Our resident Djinn
[e^]	Either *e* or ^	Look up ^ now

Carat as first character in [] negates the list

Note

- Special characters $(., *, +, ?)$ lose their special meaning inside []
- Carat means negation only when it's first in []

Regular expressions

Pattern	Expansion	Matches	Examples
`\d`	`[0-9]`	Any digit	Latitude 59
`\D`	`[^0-9]`	Any non-digit	Blue moon
`\w`	`[a-zA-Z0-9_]`	Any alphanumeric or _	Daiyu
`\W`	`[^\w]`	Not alphanumeric or _	Look!
`\s`	`[\r\t\n\f]`	Whitespace	Look␣up
`\S`	`[^\s]`	Not whitespace	Look up

Convenient aliases

Regular expressions

Groundhog is another name for woodchuck!

Pattern	Matches
`groundhog|woodchuck`	woodchuck
`yours|mine`	yours
`a|b|c`	Abba
`[gG]roundhog|[wW]oodchuck`	Woodchuck



Using the pipe symbol | for disjunction

Regular expressions

Pattern	Matches	Examples
`beg.n`	Any character	begin, begun, beg3n, beg n
`woodchucks?`	Optional s	woodchuck, woodchucks
`to*`	0 or more *o*	t, to, too, tooo
`to+`	1 or more *o*	t, to, to, too, tooo

Wildcards, optionality, repetition. ?, * and + apply to the previous character

Regular expressions

Pattern	Matches
`^[A-Z]`	Leuven
`^[^A-Za-z]`	1
`\. $`	The end.
`. $`	The end?

Begin (^) and end ($) anchors

Outline

Introduction

Regular expressions
- Basics
- Applications

Word tokenzation and normalization and sentence segmentation
- Word tokenization
- Word normalization
- Sentence segmentation

Subword tokenization
- BPE
- Subword tokenization: limitations

Summary

Regular expressions

Iterative process:

Find me all instances of the word "the" in a text.

- `the`

  Misses capitalized examples

- `[tT]he`

  Incorrectly returns `other` or `Theology`

- `\W[tT]he\W`

Regular expressions

False positives and false negatives

This process is based on fixing two kinds of errors:

- Not matching things that we should have matched (The)

  False negatives

- Matching strings that we should not have matched (there, then, other)

  False positives

Regular expressions

False positives and false negatives

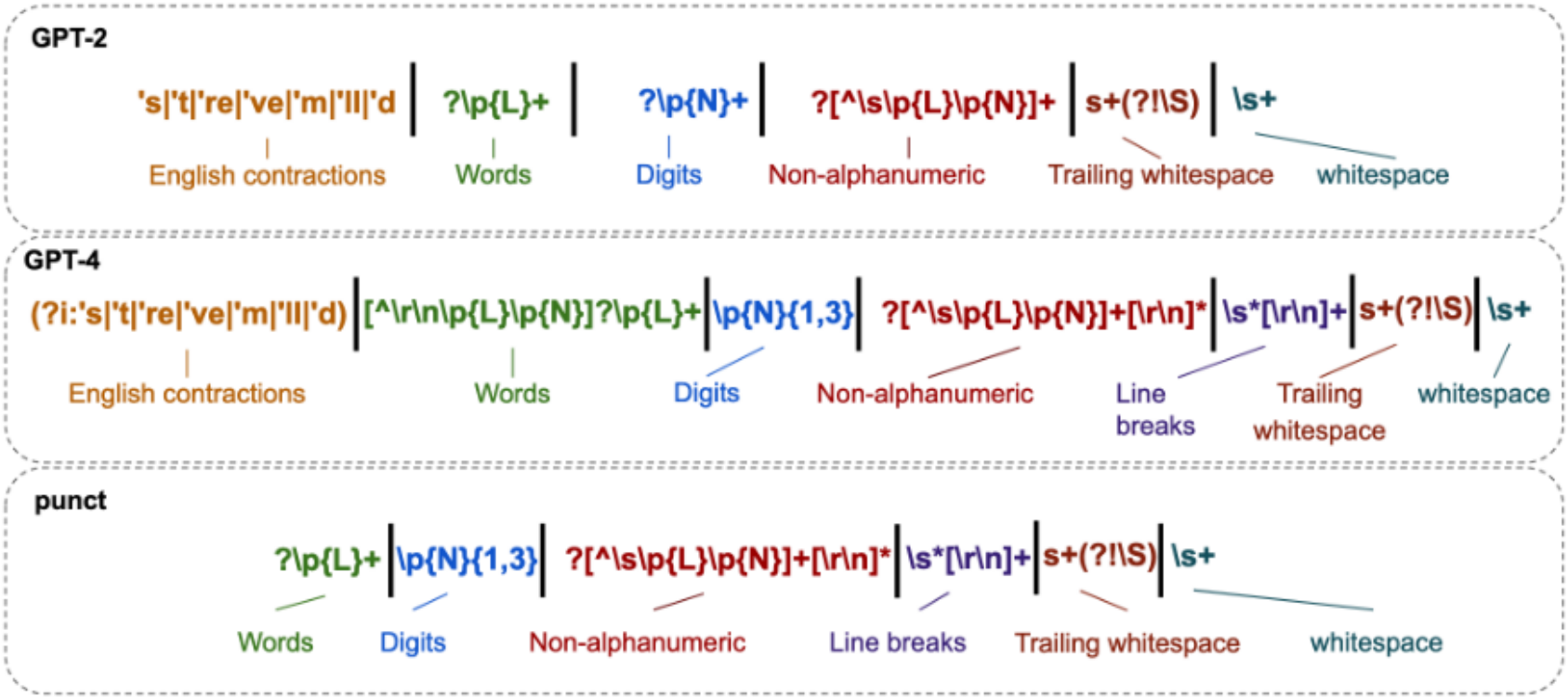
The process involves two antagonistic efforts in reducing the error rate:

- Increasing coverage (or *recall*) (minimizing false negatives).
- Increasing accuracy (or *precision*) (minimizing false positives)

This is characteristic of much work in NLP

Note: If you do not know about the recall, precision and F1 metrics, check the extra videos and slides.

Regular expressions



Regex used by different LLMs (Dagan et al., 2024)

Regular expressions

Use case: convert a text from AmE to BrE.

Substitution

Python or UNIX commands

- `s/regexp1/pattern/`
- `s/color/colour/`

Regular expressions

Use case: put angles around all numbers
e.g. *the 35 boxes* $\rightarrow$ *the <35> boxes*

Group capturing

- Parentheses () capture a pattern into a numbered register (1,2,3)
- Use `\1` to refer to the contents of the register:

```
s/([0-9]+)/<\1>/
```

Regular expressions

Group capturing: multiple registers

```
/the (.*)er they (.*), the \1er we \2/
```

- Matches

  the faster they ran, the faster we ran

- But not

  the faster they ran, the faster we ate

Regular expressions

Non-capturing groups

Add ? : after the opening parenthesis:

```
/(?:some|a few) (people|cats) like some \1/
```

- some cats like some cats

  matches

- some cats like some some

  does not match

Regular expressions

Use case: match the letter `q` but only if it is not followed by the letter `u` (match `q` in `faqir` but not in `question`)

Lookahead assertions

- `(?! pattern)` (negative lookahead): is true if pattern does not match , but is zero-width: it doesn't advance the character pointer

  `/q(?!u)/`

- `(?= pattern)` (positive lookahead): is true if pattern matches, but is zero-width

Note: the textbook is short and unclear on this topic, this link is better, or this video. We'll also see examples in the lab.

Regular expressions: application

```
  Welcome to
                 EEEEEE  LL      IIII   ZZZZZZ   AAAAA
                 EE      LL       II       ZZ   AA   AA
                 EEEEE   LL       II     ZZZ    AAAAAAA
                 EE      LL       II    ZZ      AA   AA
                 EEEEEE  LLLLLL  IIII ZZZZZZ    AA   AA

  Eliza is a mock Rogerian psychotherapist.
  The original program was described by Joseph Weizenbaum in 1966.
  This implementation by Norbert Landsteiner 2005.


ELIZA: Is something troubling you ?
YOU:   Men are all alike.
ELIZA: What is the connection, do you suppose ?
YOU:   They're always bugging us about something or other.
ELIZA: Can you think of a specific example ?
YOU:   Well, my boyfriend made me come here.
ELIZA: Is it important to you that your boyfriend made you come here ?
YOU:   He says I'm depressed much of the time.
ELIZA: I am sorry to hear that you are depressed.
YOU:   It's true. I am unhappy.
ELIZA: Can you explain what made you unhappy ?
YOU:
```

Example conversation with one of the first chatbots, Eliza 🔗

Regular expressions: application

ELIZA

```
s/.* I' M (depressed|sad) .*/I AM SORRY TO HEAR YOU ARE \1/
s/.* I AM (depressed|sad) .*/WHY DO YOU THINK YOU ARE \1/
s/.* all .*/IN WHAT WAY?/
s/.* always .*/CAN YOU THINK OF A SPECIFIC EXAMPLE?/
```

Outline

Introduction

Regular expressions
- Basics
- Applications

Word tokenzation and normalization and sentence segmentation
- Word tokenization
- Word normalization
- Sentence segmentation

Subword tokenization
- BPE
- Subword tokenization: limitations

Summary

Outline

Introduction

Regular expressions
- Basics
- Applications

Word tokenzation and normalization and sentence segmentation
- Word tokenization
- Word normalization
- Sentence segmentation

Subword tokenization
- BPE
- Subword tokenization: limitations

Summary

Word tokenization

The goose is in the sky .
The eagle in the sky has wings .
The bee in the sky has wings .
The goose with wings in the sky .
The eagle with wings in the sky .
The bee with wings in the sky .
The goose and the eagle with wings are in the sky .
The goose flies in the sky .
The wings of the bee are good wings .
The helicopter , the drone and the rocket are in the sky .
The helicopter in the sky is a useful engine .
The drone in the sky is an engine that takes pictures .
A rocket is an engine .
The helicopter is an engine .
A drone is an engine that takes pictures .
A rocket is a useful engine among sky engines .
A helicopter is flying in the sky .
The sky is full of helicopters and drones .
The engine here is a drone , the one there is a rocket .

Toy corpus

- Add spaces between punctuation: `s/([\.,])/ \1 /g` (or remove it entirely)
- Split on white spaces

Word tokenization

Problems with blindly removing punctuation

- hashtags (#nlproc)
- dates (8/09)
- URLs (kuleuven.be)
- Abbreviations (Ph.D)
- Prices (2189.54€)
- Email addresses (someone@kuleuven.be)

Word tokenization

Other difficulties

- Clitic: a word that doesn't stand on its own

  are in we're, French je in j'ai, le in l'honneur

- When should multiword expressions (MWE) be words?

  New York, rock 'n' roll

Word tokenization

Tokenization in NLTK Bird and Loper (2004)

```
>>> import nltk
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)      # set flag to allow verbose regexps
      (?:[A-Z]\.)+         # abbreviations
    | \w+(?:-\w+)*         # words with optional internal hyphens
    | \$?\d+(?:\.\d+)?%?   # currency and percentages
    | \.\.\.               # ellipsis
    | [][.,;"'?():_`-]     # these are separate tokens
  '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

Word tokenization

姚明进入总决赛 (Yao Ming reaches the finals)

- 3 words:

姚明	进入	总决赛
YaoMing	reaches	finals

- 5 words:

姚	明	进入	总	决赛
Yao	Ming	reaches	overall	finals

- 7 characters (don't use words at all):

姚	明	进	入	总	决	赛
Yao	Ming	enter	enter	overall	decision	game

Word tokenization

Writing systems that do not separate words with spaces

- Chinese: common to treat characters (hanzi) as a token

  $\Rightarrow$ segmentation is simple.

- Other languages (e.g. Thai, Japanese) require more complex word segmentation

  Supervised neural sequence models.

Outline

Introduction

Regular expressions
- Basics
- Applications

Word tokenzation and normalization and sentence segmentation
- Word tokenization
- Word normalization
- Sentence segmentation

Subword tokenization
- BPE
- Subword tokenization: limitations

Summary

Word normalization

Tokenized text

```
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40']
```

We might not want to create separate word vectors for

- That vs that
- U.S.A vs USA
- costs vs cost
- 12.40 vs 12.41

Word normalization

Case folding

- Reducing all letters to lower case may be useful in some applications, e.g. IR where users tend to use lower case
- Possible exception: upper case in mid-sentence
  - Apple vs apple
  - March vs march
  - Turkey vs turkey
- Case is helpful in other applications, e.g. MT or IE
  - US vs us

Word normalization

Lemmatization

Represent all words as their lemma, their shared root.

- `am, are, is` → `be`
- `car, cars, car's, cars'` → `car`
- Spanish `quiero` *(I want)*, `quieres` *(you want)* → `querer` *(want)*
- `He is reading detective stories` → `He be read detective story`

See more in lecture 7.

Outline

Introduction

Regular expressions
- Basics
- Applications

Word tokenzation and normalization and sentence segmentation
- Word tokenization
- Word normalization
- Sentence segmentation

Subword tokenization
- BPE
- Subword tokenization: limitations

Summary

Sentence segmentation

Split by punctuation

!, ? mostly unambiguous but period "." is very ambiguous

- Sentence boundary
- Abbreviations like Inc. or Dr.
- Numbers like .02% or 4.3

Two-step procedure

1. Classify periods as part of the word or sentence boundary
   - Using rules
     Using a regex or an abbreviation dictionary
   - Using ML
2. Based on this, segment sentences by rules.

Outline

Introduction

Regular expressions
- Basics
- Applications

Word tokenzation and normalization and sentence segmentation
- Word tokenization
- Word normalization
- Sentence segmentation

Subword tokenization
- BPE
- Subword tokenization: limitations

Summary

Levels of tokenization

Sample Data:

"This is tokenizing."

+: small vocabulary -: long sequences

Character Level

[T] [h] [i] [s] [i] [s] [t] [o] [k] [e] [n] [i] [z] [i] [n] [g] [.]

Word Level

[This] [is] [tokenizing] [.]

+: short sequences
-: large vocabulary
(or coverage issues:
out-of-vocabulary words)

Subword Level

[This] [is] [token] [izing] [.]

Good trade-off (for English)



Subword tokenization

Subwords

Determined in a data-driven way, using a compression algorithm.
Common ones:

- Byte-Pair Encoding (BPE; Sennrich et al. (2016))
- Unigram language model (Kudo, 2018)
- WordPiece (Schuster and Nakajima, 2012)

Subword tokenizers components

- Token learner: induces a *vocabulary* from a training corpus.
- segmenter: tokenizes a raw (test) sentence according to that vocabulary.

Outline

Introduction

Regular expressions
- Basics
- Applications

Word tokenzation and normalization and sentence segmentation
- Word tokenization
- Word normalization
- Sentence segmentation

Subword tokenization
- BPE
- Subword tokenization: limitations

Summary

Subword tokenization

BPE algorithm

Let the vocabulary be the set of all individual characters
`{A, B, C, D,⋯, a, b, c, d⋯.}`

Repeat:

Choose the two symbols that are most frequently adjacent in the training corpus (say `A`, `B`)
Add a new merged symbol `AB` to the vocabulary
Replace every adjacent `A B` in the corpus with `AB`.

Until k merges have been done.

Subword tokenization

BPE: notes

- Most subword algorithms are run inside space-separated tokens ⇒ before training:
  - Add a special end-of-word symbol (e.g. '_') before spaces in the training corpus
  - Separate into letters.
- For efficiency, bigrams that cross word boundaries are not considered

BPE token learner

Corpus

low low low low low lowest lowest newer newer newer newer newer newer wider wider wider new new

Corpus

```
5  l o w _
2  l o w e s t _
6  n e w e r _
3  w i d e r _
2  n e w _
```

Vocabulary

_, d, e, i, l, n, o, r, s, t, w

BPE token learner

Corpus

```
5  l o w _
2  l o w e s t _
6  n e w er_
3  w i d er_
2  n e w _
```

Vocabulary

_, d, e, i, l, n, o, r, s, t, w, er, er_

First merge

e r → er

Second merge

er _ → er_

BPE token learner

Corpus

```
5  l o w _
2  l o w e s t _
6  ne w er_
3  w i d er_
2  ne w _
```

Vocabulary

_, d, e, i, l, n, o, r, s, t, w,
er, er_, ne

Third merge

n e → ne

BPE token learner

Merges 4 to 8

Merge	Vocabulary
(ne, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new
(l, o)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo
(lo, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low
(new, er_)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_
(low, _)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_, low_

BPE token segmenter

On the test data, run each merge learned from the training data:

- Greedily
- In the order we learned them

$\Rightarrow$ merge every `e r` to `er`, then merge `er _` to `er_`, etc.

Result

- `n e w e r _` would be tokenized as a full word
- `l o w e r _` would be two tokens: `low er_`

Outline

Introduction

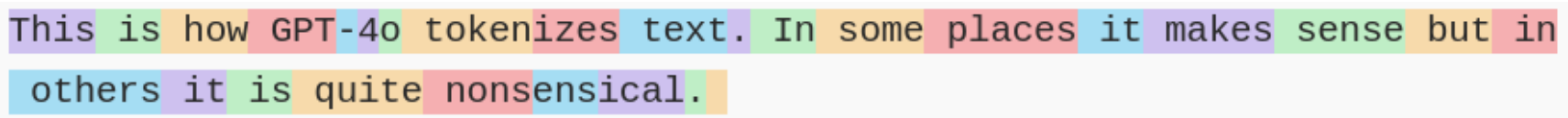
Regular expressions
- Basics
- Applications

Word tokenzation and normalization and sentence segmentation
- Word tokenization
- Word normalization
- Sentence segmentation

Subword tokenization
- BPE
- Subword tokenization: limitations

Summary

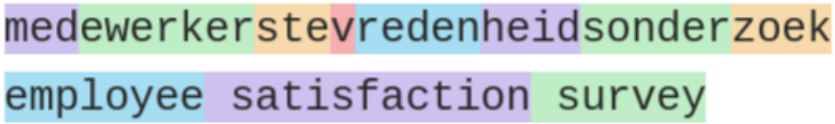
Subword tokenization: limitations



Example tokenization result by GPT-4o. 🔗

- Subwords are determined based on frequency, they are not (necessarily) linguistically meaningful.

Subword tokenization: limitations



Example tokenization result by GPT-4o.

- Subwords are determined based on frequency, they are not (necessarily) linguistically meaningful.
- Subwords are suboptimal for morphologically complex languages (Klein and Tsarfaty, 2020)
- The vocabulary of multilingual LLMs is dominated by English.
- This leads to longer sequences for non-English languages, which is more costly and less accurate (Ahia et al., 2023).

Subword tokenization: addressing the limitations

- Subwords are determined based on frequency, they are not (necessarily) linguistically meaningful.

  Subword tokenizers can be modified to be informed by morphology (see lecture 7)

- Subwords are suboptimal for morphologically complex languages (Klein and Tsarfaty, 2020)

  Alternative units of representation? (next slide)

- The vocabulary of multilingual LLMs is dominated by English.
- This leads to longer sequences for non-English languages, which is more costly and less accurate (Ahia et al., 2023).

  Allocate vocabulary more fairly across languages (e.g. Foroutan et al. (2025))

Subword tokenization: addressing the limitations

Alternatives to subwords

- Character-based models (Lee et al., 2017)
- Byte-based models (Wang et al., 2020)
- Pixel-based models (Rust et al., 2023)

Limitations

Characters and bytes lead to long sequences. Pixel-based representations lag behind subwords in terms of accuracy. *(But we are working on bridging this gap!)*

Subword tokenization and alternatives

- A long ignored but increasingly active area of research
- Recently got its own workshop TokShop
- Nice invited talks with videos online (if you are curious):
  Yuval Pinter overview of the evolution of tokenization 🎥🔗
  Desmond Elliott Pixel-based models 🎥🔗

Outline

Introduction

Regular expressions
- Basics
- Applications

Word tokenzation and normalization and sentence segmentation
- Word tokenization
- Word normalization
- Sentence segmentation

Subword tokenization
- BPE
- Subword tokenization: limitations

Summary

Summary

Regular expressions

- are a useful tool for language processing,
- were previously the main technique for tokenization,
- are still used nowadays for pre-processing (or pre-tokenization) and normalization, prior to subword tokenization,
- are still used for sentence segmentation.

Subwords

- are learned in a data-driven way, with a compression algorithm,
- have limitations.
- Overcoming these limitations is an open challenge.

References

Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David Mortensen, Noah Smith, and Yulia Tsvetkov. 2023. Do all languages cost the same? tokenization in the era of commercial language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.

Steven Bird and Edward Loper. 2004. NLTK: The natural language toolkit. In *Proceedings of the ACL Interactive Poster and Demonstration Sessions*.

Gautier Dagan, Gabriel Synnaeve, and Baptiste Rozière. 2024. Getting the most out of your tokenizer for pre-training and domain adaptation.

Negar Foroutan, Clara Meister, Debjit Paul, Joel Niklaus, Sina Ahmadi, Antoine Bosselut, and Rico Sennrich. 2025. Parity-aware byte-pair encoding: Improving cross-lingual fairness in tokenization.

Daniel Jurafsky and James H. Martin. 2024. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*, 3rd edition. Online manuscript released August 20, 2024.

Stav Klein and Reut Tsarfaty. 2020. Getting the ##life out of living: How adequate are word-pieces for modelling complex morphology? In *Proceedings of the 17th SIGMORPHON Workshop on Computational Research in Phonetics, Phonology, and Morphology*.

Taku Kudo. 2018. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Jason Lee, Kyunghyun Cho, and Thomas Hofmann. 2017. Fully character-level neural machine translation without explicit segmentation. *Transactions of the Association for Computational Linguistics*, 5:365–378.

Phillip Rust, Jonas F. Lotz, Emanuele Bugliarello, Elizabeth Salesky, Miryam de Lhoneux, and Desmond Elliott. 2023. Language modelling with pixels. In *The Eleventh International Conference on Learning Representations*.

Mike Schuster and Kaisuke Nakajima. 2012. Japanese and korean voice search. In *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Changhan Wang, Kyunghyun Cho, and Jiatao Gu. 2020. Neural machine translation with byte-level subwords. In *AAAI*.